

第01章

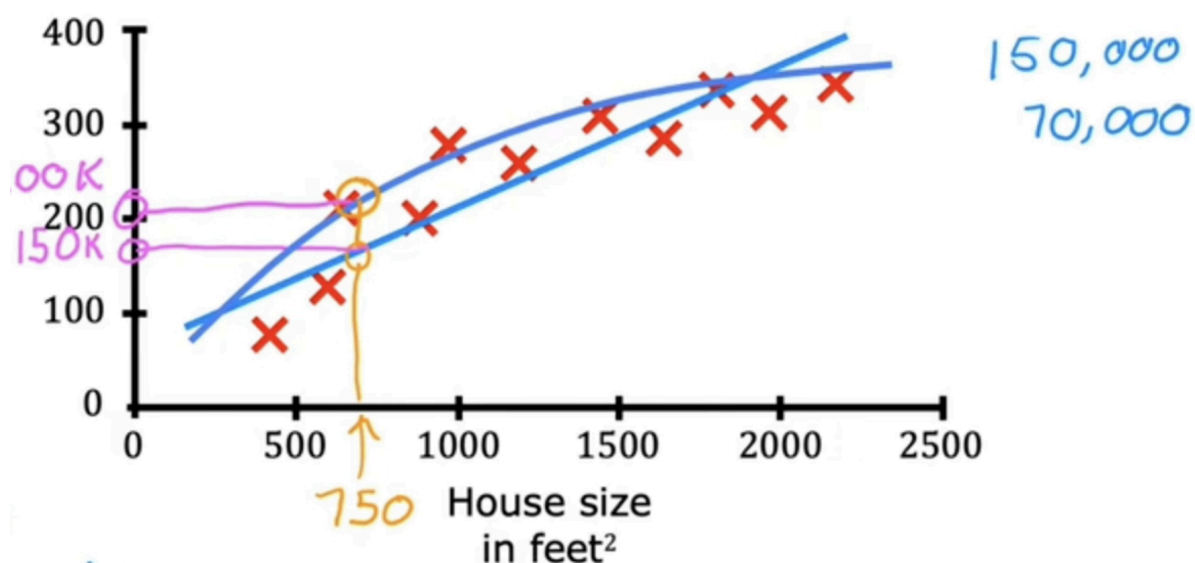
class

supervised learning

Regression回归

Predict a number infinitely many possible outputs.

on: Housing price prediction 利用规律进行预测?



Classification分类

predict categories small number of possible outputs.

- benign
- × malignant type1
- △ malignant type 2



unsupervised learning

data only comes with inputs x , but not output labels y . Algorithm has to find structure in the data

Clustering

Group similar data points together

#eg : Google news、DNA microarray、Grouping customers

Dimensionality reduction

Compress data using fewer numbers

Anomaly detection

Find unusual data points

linear regression

Training: Data used to train the model

x = "input" variable feature

y = "output" variable/ "target" variable

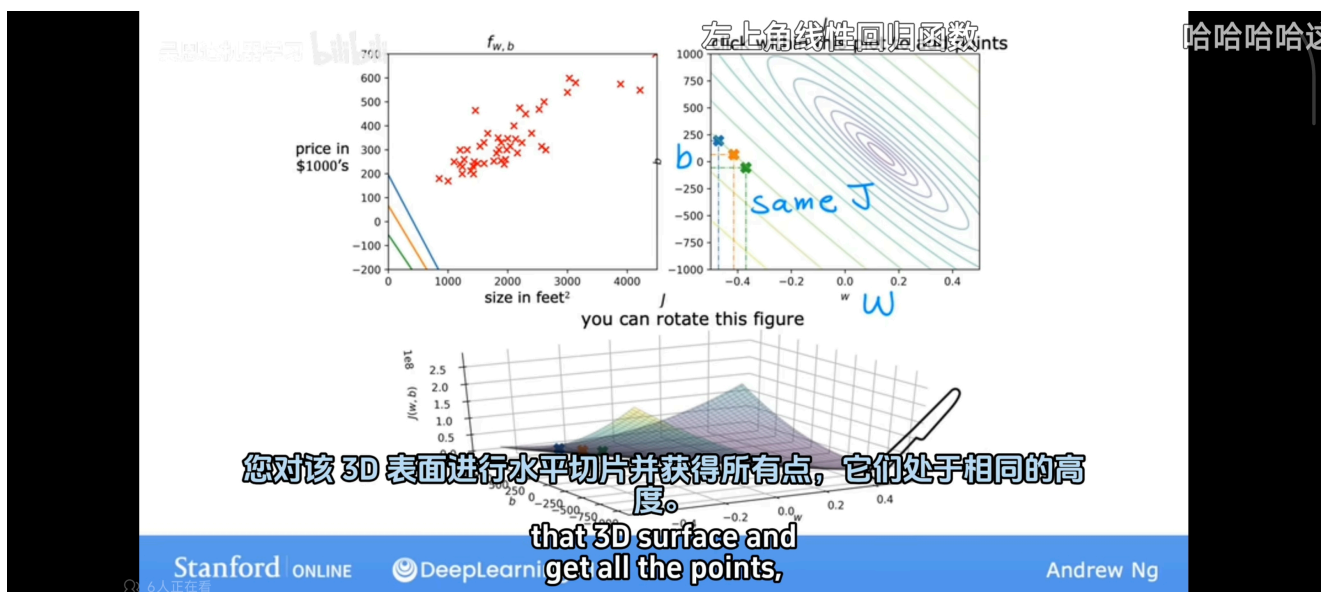
m = number of training examples

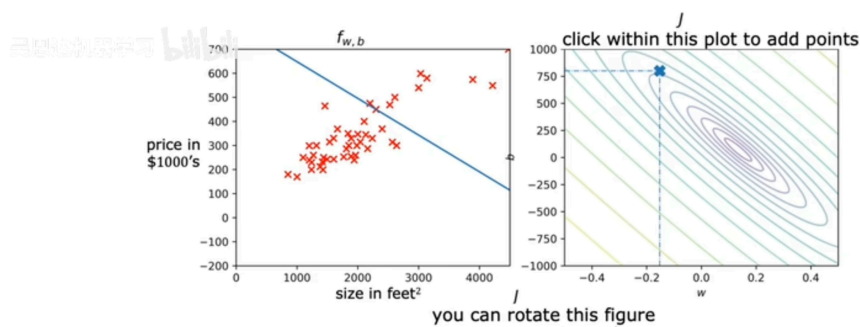
$(x^{(i)}, y^{(i)}) = i^{th} \text{ training examples}$

Model: $f_{w,b}(x) = wx + b$, (w, b named parameters or coefficients or weights)

[1]

$$\text{Cost function : } J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m (f_{(w,b)}(x^{(i)}) - y^{(i)})^2$$





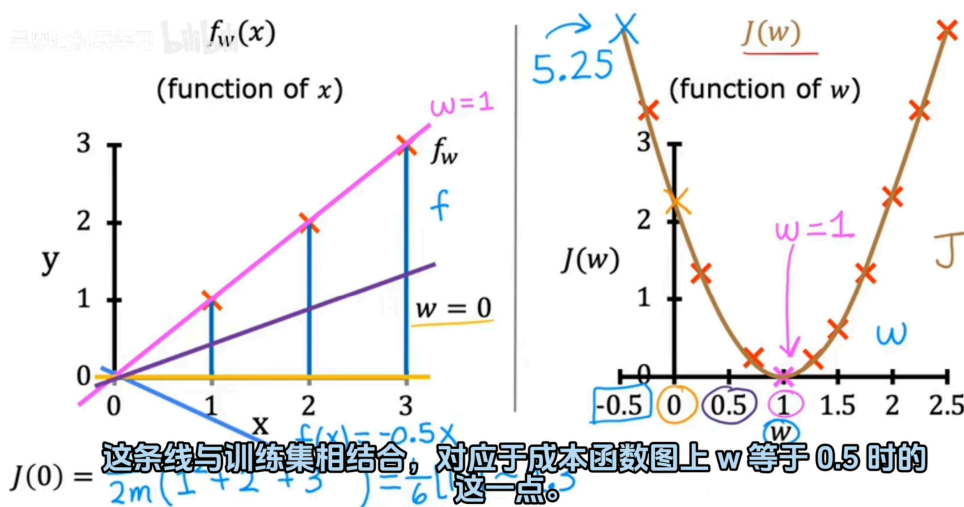
在这里，您在图 J 上有一个特定点。

Over here, you have a particular point on the graph j.

Stanford ONLINE

DeepLearning.AI

Andrew Ng



这条线与训练集相结合，对应于成本函数图上 w 等于 0.5 时的这一点。

the training set corresponds to

Stanford ONLINE

DeepLearning.AI

Andrew Ng

Gradient descent algorithm

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b) = w - \alpha \frac{d}{dw} J(w), \alpha \text{ is learning rate}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

Near a local minimum,

- derivative becomes smaller
 - update steps become smaller
- Can reach minimum without decreasing learning rate

repeat until convergence:

$$w = w - \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

"Batch" gradient descent

Each step of gradient descent uses all the training examples

multiple linear regression

$$f_{\vec{w},b} = \vec{w} \cdot \vec{x} + b = w_1x_1 + w_2x_2 + w_3x_3 \dots$$

#Vectorization can faster the process

f=0

for f in range(n):

 f=f+w[j]*x[j]

f=f+b

f=np.dot(w,x)+b

w=w-0.1*d

PYTHON

Gradient descent:

repeat:

$$w = w - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

An alternative to gradient descent

Normal equation

- Only for linear regression
- Solve for w,b without iterations

Disadvantages

- Doesn't generalize to other learning algorithm
- Slow when number of features is large(>10000)

normalization

Feature scaling

eg $x_1 < 2000$, $x_{1,scaled} = \frac{x_1}{2000}$

Mean normalization

[2]

eg $300 \leq x_1 \leq 2000, x_1 = \frac{x_1 - u_1}{2000 - 300}$

Z-score normalization

[3]

eg $x_1 = \frac{x_1 - u_1}{\sigma_1}$

skills

make values of α to be 3 times of itself

eg 0.001, 0.003, 0.01, 0.03

Feature engineering

Using intuition to design new features, by transforming or combining original feature

eg $x_3 = x_1 * x_2$

Polynomial regression

$$f_{\vec{w},b}(x) = w_1x + w_2x^2 + b$$

feature scaling: eg add x^3 to avoid the line from go down

Classification

"binary classification"

logistic regression

sigmoid function: outputs between 0 and 1

$$g(x) = \frac{1}{1 + e^{-x}}, 0 < g(x) < 1$$

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-\vec{w} \cdot \vec{x} + b}}$$

Decision Boundary

eg. $z = x_1 + x_2 - 3$, easy to get $x_1 + x_2 = 3$

Non-linear decision boundaries

eg. $z = x_1^2 + x_2^2 - 1$

Cost

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\vec{w}, b}(\vec{x}^{(i)}, y^{(i)}) = \begin{cases} -\log(f_{\vec{w}, b}(\vec{x}^{(i)})), & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})), & \text{if } y^{(i)} = 0 \end{cases}$$

4在这

序运行的速度，特别是在循环里。

Simplified cost function

1/4

损失函数是单个，代价函数相当于整体

4

$$L(f_{\vec{w}, b}(\vec{x}^{(i)}, y^{(i)}) = -y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)}))$$

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m [L(f_{\vec{w}, b}(\vec{x}^{(i)}, y^{(i)})]$$

$$= \frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)}))]$$

几乎所有人用来训练逻辑回归的代价函数

The cost function that pretty much everyone uses to train logistic regression.

Stanford ONLINE

DeepLearning.AI

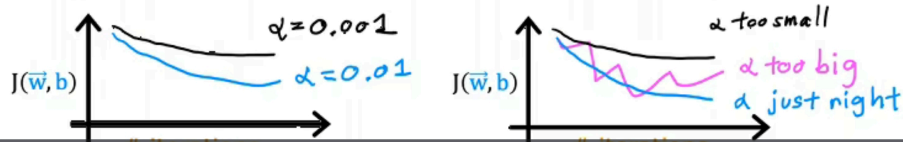
Andrew Ng

不规范~不管阿尔法是多少，最后收敛到的值都应该是同一个值

Values of α to try:

这个能不图没问题，确实收

... 0.001 0.003 0.01 0.03 0.1 0.3 1 ...
 $\nearrow$ $\nearrow$ $\nearrow$ $\nearrow$ $\nearrow$ $\nearrow$
 $3\times$ $\approx 3\times$ $3\times$ $\approx 3\times$ $3\times$ $\approx 3\times$



所以我希望这个技巧也能帮助你为你的梯度实现选择一个好的学习率
 So I hope this technique too will be useful for you to choose a good learning rate for your implementation of gradient

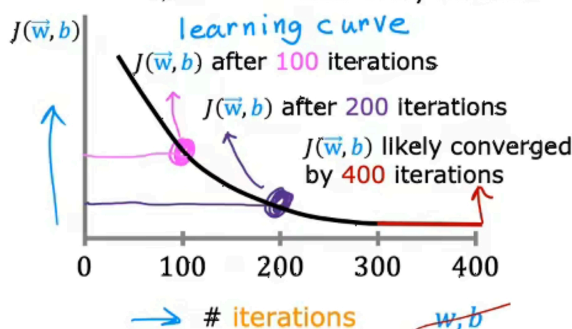
Stanford ONLINE

DeepLearning.AI

Andrew Ng

Make sure gradient descent is working correctly

objective: $\min_{\vec{w}, b} J(\vec{w}, b)$ $J(\vec{w}, b)$ should decrease after every iteration



Automatic convergence test

Let ϵ "epsilon" be 10^{-3} .
 0.001

If $J(\vec{w}, b)$ decreases by $\leq \epsilon$ in one iteration, declare convergence.

(found parameters $\vec{w}, b$ to get close to global minimum)

iterations needed varies 30 1,000 100,000

Stanford ONLINE

DeepLearning.AI

Andrew Ng

book

learning training data sample set hypothesis learner

classification

二分类任务: $y=\{-1,+1\}$ 或 $\{0, 1\}$

多分类任务: $|y|>2$

回归任务, $y=\mathbb{R}$

误差

$$E_{ote}(\mathfrak{L}_a|X, f) = \sum_h \sum_{x \in \mathcal{X}-X} P(x) \mathbb{I}(h(x) \neq f(x)) P(h | X, \mathfrak{L}_a) ,$$

误差与学习方法无关

$$\sum_f E_{ote}(\mathfrak{L}_a|X, f) = \sum_f E_{ote}(\mathfrak{L}_b|X, f) ,$$

-
1. 成本函数 $\hookrightarrow$
 2. 均值归一化 $\hookrightarrow$
 3. 标准差 $\hookrightarrow$